INSA TOULOUSE

Virtualisation et Cloud Computing

2026–2027

Evelyne AKOPYAN
evelyne.akopyan@insa-toulouse.fr

TP VCC-1

De la machine au conteneur

Objectifs pédagogiques

À l'issue de cette séance, vous devrez être capables de :

- expliquer la différence entre une machine virtuelle et un conteneur
- créer et administrer une machine virtuelle simple
- exécuter une application dans un conteneur Docker
- construire une image Docker reproductible à l'aide d'un Dockerfile
- déployer une application multi-conteneurs avec Docker Compose
- organiser proprement les fichiers produits dans un dépôt Git

Contexte

Vous faites partie d'une jeune startup qui développe une application distribuée. Votre travail consiste à rendre cette application **portable**, c'est-à-dire capable de s'exécuter de manière identique sur différentes machines.

Au cours de cette séance, vous découvrirez deux technologies fondamentales :

1. la **virtualisation**, qui permet d'exécuter plusieurs systèmes d'exploitation sur une même machine physique
2. les **conteneurs**, qui permettent d'encapsuler une application et toutes ses dépendances afin d'en faciliter le déploiement.

Organisation du dépôt Git

Les fichiers produits lors de chaque séance seront réutilisés dans les TP suivants. Votre dépôt Git est donc votre principal support de travail. Nous vous recommandons l'organisation suivante :

```
vcc-labs/
|
+-- README.md
|
+-- TP1/
|   +-- Dockerfile
|   +-- docker-compose.yml
|   +-- README.md
|   +-- ...
|
+-- TP2/...
```

Le dépôt devra être partagé entre les membres du binôme.

Le fichier **README.md** situé à la racine du dépôt présentera brièvement le projet et/ou l'organisation du dépôt. Chaque TP doit posséder son propre dossier contenant les fichiers produits durant la séance, un README succinct expliquant le travail réalisé, et les éventuelles captures d'écran et autres fichiers.

1 Découverte de la virtualisation

Objectif

Créer et utiliser une première machine virtuelle.

1.1 Création de la machine virtuelle

Lancez l'application **VirtualBox** et **importez** une nouvelle machine virtuelle (VM). Les caractéristiques de la machine sont les suivantes :

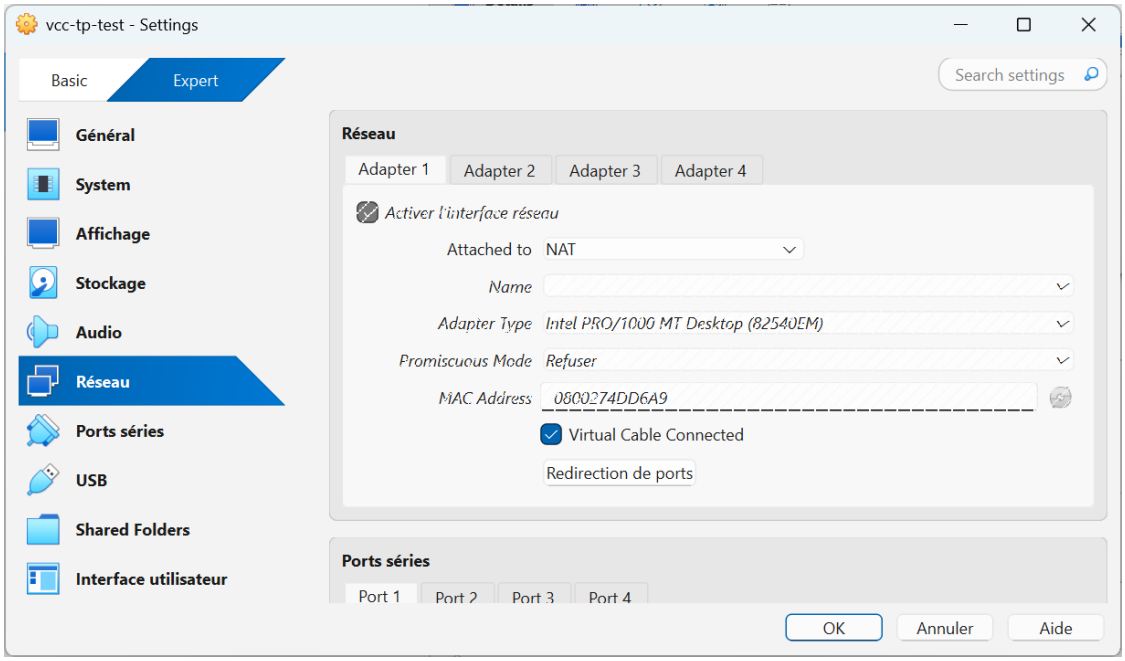
Paramètre	Valeur
Instance OVA (VM)	Lien de téléchargement
OS	Linux Ubuntu (64-bit)
Mémoire vive	2048 MB
Processeur	1 CPU
Disque	25 Gio (VDI)
Login	insa
Mot de passe	insa

Démarrez ensuite la VM.

1.2 Configuration réseau

Une machine virtuelle est, par défaut, isolée de la machine hôte. Afin de pouvoir communiquer avec elle, il est nécessaire de configurer correctement son interface réseau.

Un serveur **OpenSSH** est installé sur la VM. Configurez la VM afin qu'elle soit accessible depuis votre poste de travail en SSH depuis le **port 5555**. Une redirection de port peut être nécessaire depuis le panneau de configuration de VirtualBox.



1.3 Premier accès SSH

Depuis votre machine hôte, connectez-vous à la VM en SSH. Rappel de la commande SSH depuis une machine hôte :

```
ssh login@host -p numport
```

Une fois connecté, vérifiez la version d'Ubuntu et affichez votre adresse IP. Vous pouvez vous servir des commandes :

```
vcc-version
ifconfig
```

1.4 Observation de l'environnement

Prenez le temps d'identifier votre environnement, notamment le nombre de CPU visibles par le système, l'utilisation de la RAM et l'espace disque.

```
nproc
top
df -h
```

1.5 Checkpoint

Dans le fichier "TP1/README.md", ajoutez une section "Etape 1" et répondez aux questions suivantes :

1. Expliquez, avec vos propres mots, ce qu'est une machine virtuelle.
2. Citez deux avantages de la virtualisation dans un environnement professionnel.
3. Quelle différence existe-t-il entre travailler directement sur votre ordinateur et travailler dans une machine virtuelle ?

2 Découverte des conteneurs

Objectif

Découvrir le fonctionnement des conteneurs Docker, et comprendre en quoi ils diffèrent des machines virtuelles.

2.1 Installation de Docker

Installez Docker sur votre machine virtuelle Ubuntu. Pour cela, commencez par ajouter la clé GPG officielle de Docker :

```
sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/
    docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc
```

Puis ajoutez le répertoire aux sources **apt** :

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
```

```
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

Enfin, installez la dernière version des packages Docker :

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
    compose-plugin
```

Une fois l'installation terminée, vérifiez que le service Docker est démarré :

```
systemctl status docker
```

2.2 Premier conteneur

Vérifiez que l'installation est correcte : pour cela, lancez un conteneur Docker générique.

```
sudo docker run hello-world
```

Observez les différentes étapes : le téléchargement de l'image, la création et l'exécution du conteneur.

Une fois le programme terminé, vérifiez la liste des conteneurs présents sur votre machine :

```
sudo docker container ls --all
```

Quelques commandes utiles pour Docker :

```
sudo docker start <container-name>
sudo docker exec -it <container-name> bash
sudo docker docker-subcommand --help
```

2.3 Images et conteneurs

Docker permet de créer des conteneurs à partir d'images (disponibles sur Docker Hub). Pour créer votre propre conteneur, vous allez commencer par récupérer une image (par exemple "ubuntu"), puis créer un conteneur (nommé "cnt1") à partir de cette image :

```
sudo docker pull ubuntu
sudo docker run --name cnt1 -it ubuntu
```

Une fois le conteneur créé, il faut y installer des packages pour pouvoir tester sa connectivité :

```
apt -y update
apt -y install net-tools iputils-ping
apt -y upgrade
```

Puis :

— Affichez l'adresse IP du conteneur.
— Testez la connectivité à Internet en faisant un **ping** vers ubuntu.com.
— Faites un ping du conteneur vers la VM, puis de la VM vers le conteneur. Qu'observez-vous ?

2.4 Checkpoint

Dans le fichier "TP1/README.md", ajoutez une nouvelle section "Étape 2" et répondez aux questions suivantes :

1. Expliquez, avec vos propres mots, ce qu'est un conteneur Docker.
2. Quelle différence fondamentale existe entre une machine virtuelle et un conteneur ?
3. Pourquoi les conteneurs sont-ils particulièrement adaptés au déploiement d'applications dans le Cloud ?

Pour aller plus loin

Docker Hub contient plusieurs milliers d'images prêtes à l'emploi. Choisissez une image de votre choix (base de données, serveur Web, outil de supervision, etc.). Déployez-la localement et identifiez son rôle, les paramètres nécessaires à son exécution ainsi que les ports exposés.

3 Construire une image reproductible

Objectif

Construire une image Docker à l'aide d'un Dockerfile afin de rendre le déploiement d'une application entièrement reproductible.

3.1 Récupération de l'application

Placez-vous dans l'espace de travail de votre VM et clonez le dépôt contenant les applications du TP :

```
cd ~/workspace
git clone https://github.com/eakopyan/insa-labs.git
```

Ensuite, ouvrez le dossier de l'application **hello-api**, et observez son contenu, notamment les dépendances et le port utilisé.

3.2 Création du Dockerfile

Comme mentionné en cours, Docker permet de construire une image à partir d'un fichier nommé **Dockerfile**. Créez ce fichier dans le dossier de l'application :

```
touch Dockerfile
```

Vous allez maintenant le compléter progressivement.

3.2.1 Choisir une image de base

Une image Docker est toujours construite à partir d'une image existante. Ajoutez une première instruction :

```
FROM python:3.12-slim
```

3.2.2 Définir le répertoire de travail

Il faut indiquer le répertoire dans lequel seront exécutées les commandes suivantes :

```
WORKDIR /app
```

3.2.3 Copier les fichiers

L'application doit être copiée dans l'image :

```
COPY . .
```

3.2.4 Installer les dépendances

Le fichier "requirements.txt" contient les bibliothèques Python nécessaires au fonctionnement de l'application :

```
RUN pip install --no-cache-dir -r requirements.txt
```

3.2.5 Exposer le port

L'application écoute via l'exposition d'un port. Rapportez ce numéro de port dans le Dockerfile :

```
EXPOSE <NUMPORT>
```

3.2.6 Démarrer l'application

Enfin, indiquez la commande à exécuter au démarrage du conteneur :

```
CMD ["python", "app.py"]
```

Votre Dockerfile est maintenant terminé.

3.3 Création du conteneur

À partir du Dockerfile, construisez une image Docker. Choisissez un nom et un numéro de version explicites (par exemple "hello-api :1.0"). Une fois la construction terminée, vérifiez que l'image apparaît dans la liste des images Docker :

```
sudo docker build -t hello-api:1.0 .
sudo docker images
```

A présent, créez et démarrez le conteneur, puis testez l'accès à l'application :

```
sudo docker run hello-api:1.0
curl http://localhost:<NUMPORT>
```

Que se passe-t-il ? Pouvez-vous accéder à l'application avec **curl** ?

Relancez maintenant le conteneur en publiant le port d'écoute :

```
docker run -p <NUMPORT>:<NUMPORT> hello-api:1.0
```

Puis testez à nouveau l'accès à l'application. Qu'obtenez-vous ?

3.4 Checkpoint

Ajoutez une nouvelle section "Etape 3" et répondez aux questions suivantes :

1. Pourquoi un Dockerfile est-il préférable à la configuration manuelle d'un conteneur ?
2. Quelle différence existe entre une image Docker et un conteneur Docker ?

4 Déployer une application multi-conteneurs

Objectif

Déployer une application composée de plusieurs services à l'aide de Docker Compose.

4.1 Découverte de l'application

Dans une architecture orientée services, une application est généralement composée de plusieurs services qui communiquent entre eux. Mais gérer ces conteneurs un par un devient rapidement fastidieux.

L'application à déployer est une calculatrice, que vous avez cloné en même temps que la première application. Allez dans le dossier **calculator** et vérifiez son contenu. Combien y a-t-il de microservices ? Combien de Dockerfiles ?

4.2 Utilisation de Docker Compose

Docker Compose permet d'orchestrer le lancement de conteneurs. Pour cela, il a besoin d'un descriptif YAML, nommé **docker-compose.yml**. Créez ce fichier sous le répertoire "calculator". Ensuite, vous pourrez déclarer chaque service de la façon suivante :

```
services:
        addition:
                build: ./add-service
```

Pour l'interaction avec le service "calculatrice", il faudra ajouter les ports d'écoute :

```
        calculator:
                build: ./calc-service
                ports:
                        - "50000:80"
```

Le premier correspond au port de la VM ; le second est le port du conteneur.

Une fois le fichier complété, vous pouvez lancer le build avec la commande suivante :

```
sudo docker compose build
```

Vérifiez ensuite que toutes les images ont bien été générées.

4.3 Déploiement de l'application

Lancez l'application à l'aide de Docker Compose :

```
sudo docker compose up
```

Puis testez l'application dans un second terminal :

```
curl -v -X POST -d "2+3" http://localhost:50000
```

Que se passe-t-il si vous essayez de joindre le port 80 ? Pourquoi ?

4.4 Découverte du réseau

Repérez le réseau créé par Docker Compose, puis inspectez les interfaces du service "calculatrice" :

```
sudo docker network ls
sudo docker network inspect calculator_default
```

Qu'observez-vous ? Quels conteneurs appartiennent à ce réseau ?

4.5 Arrêt de l'application

Arrêtez proprement l'application :

```
sudo docker compose down
```

Puis affichez la liste des conteneurs. Existent-ils encore ?

4.6 Checkpoint

Ajoutez une dernière section "Étape 4" et répondez aux questions suivantes :

1. Pourquoi Docker Compose est-il préférable au lancement manuel de plusieurs conteneurs ?
2. Quel est le rôle du fichier "docker-compose.yml" ?
3. Dans quels cas Docker Compose pourrait-il ses limites ?